

09 — Sequences & Identity Columns in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
2. [What is a Sequence?](#)
3. [Creating Sequences](#)
4. [Sequence Functions](#)
5. [SERIAL Types \(Legacy\)](#)
6. [GENERATED AS IDENTITY \(Modern\)](#)
7. [Sequence Ownership and Dependencies](#)
8. [Gaps in Sequences](#)
9. [Resetting and Altering Sequences](#)
10. [Multi-Tenant and Distributed ID Strategies](#)
11. [SQL Examples](#)
12. [Common Mistakes](#)
13. [Best Practices](#)
14. [Performance Considerations](#)
15. [Interview Questions & Answers](#)
16. [Exercises with Solutions](#)
17. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

- Create and manage sequences with custom parameters
- Explain the difference between SERIAL and GENERATED AS IDENTITY
- Handle sequence exhaustion and migration strategies
- Use sequences safely in concurrent environments
- Design distributed-friendly ID strategies

What is a Sequence?

A **sequence** is a special database object that generates unique integer values in order.

Sequence object properties:

Name:	orders_order_id_seq
Data type:	bigint
Current:	1042
Start:	1
Increment:	1
Minimum:	1
Maximum:	9223372036854775807 (bigint max)
Cache:	1
Cycle:	no

Key properties of sequences:

- Operations on sequences are **not transactional** — a `nextval()` call is never rolled back
 - Sequences are **crash-safe** — cached values may be "lost" after a crash
 - `nextval()` is **never blocked** by concurrent transactions — no serialization
 - The "current value" is tracked **per session** with `currval()`
-

Creating Sequences

```
-- Basic sequence
CREATE SEQUENCE my_seq;

-- Full syntax with all options
CREATE SEQUENCE IF NOT EXISTS order_number_seq
    AS BIGINT          -- data type (SMALLINT, INTEGER, BIGINT)
    START WITH 1000     -- first value
    INCREMENT BY 1      -- step size (negative for descending)
    MINVALUE 1000       -- lower bound
    MAXVALUE 9999999    -- upper bound
    CACHE 50           -- pre-allocate 50 values per session
    NO CYCLE;          -- do not wrap around at MAXVALUE
    -- CYCLE would restart at MINVALUE after MAXVALUE

-- Descending sequence (countdown)
CREATE SEQUENCE countdown_seq
    START WITH 100
    INCREMENT BY -1
    MINVALUE 1
    MAXVALUE 100
    NO CYCLE;

-- View all sequences in current schema
SELECT * FROM information_schema.sequences WHERE sequence_schema = 'public';

-- Inspect a specific sequence
SELECT * FROM pg_sequences WHERE sequencename = 'order_number_seq';
```

Sequence Functions

```
-- Get next value (advances the sequence — cannot be undone!)
SELECT nextval('order_number_seq'); -- 1000
SELECT nextval('order_number_seq'); -- 1001

-- Get current value (only valid after calling nextval in current session)
SELECT currval('order_number_seq'); -- 1001

-- Set value (use with care — can cause conflicts!)
SELECT setval('order_number_seq', 5000);          -- next call returns 5001
SELECT setval('order_number_seq', 5000, false); -- next call returns 5000
```

```

-- Peek at sequence state without advancing
SELECT last_value, is_called FROM order_number_seq;
-- is_called = FALSE means: next nextval() returns last_value (not last_value+1)

-- Using sequences in queries
INSERT INTO orders (order_id, customer_id, total)
VALUES (nextval('order_number_seq'), 42, 99.99);

-- Using in DEFAULT
CREATE TABLE orders (
    order_id BIGINT DEFAULT nextval('order_number_seq') PRIMARY KEY
);

```

SERIAL Types (Legacy)

`SERIAL`, `SMALLSERIAL`, and `BIGSERIAL` are **syntactic shortcuts** that expand to a sequence + column default. They are still widely used but considered legacy in new code.

```

-- This:
CREATE TABLE t (id SERIAL PRIMARY KEY);

-- Expands to exactly:
CREATE SEQUENCE t_id_seq
    AS INTEGER
    START WITH 1
    INCREMENT BY 1
    NO MINVALUE
    NO MAXVALUE
    CACHE 1;

CREATE TABLE t (
    id INTEGER NOT NULL DEFAULT nextval('t_id_seq')
);

ALTER SEQUENCE t_id_seq OWNED BY t.id;
ALTER TABLE t ADD CONSTRAINT t_pkey PRIMARY KEY (id);

```

SERIAL problems

```

-- Problem 1: Permissions – must grant sequence separately
GRANT USAGE ON SEQUENCE t_id_seq TO app_user; -- easy to forget!

-- Problem 2: Not SQL standard – IDENTITY is standard

-- Problem 3: Sequence is "detached" – tools may not include it in backups/exports

-- Problem 4: Cannot use ALWAYS (prevent manual insert)
-- With SERIAL, anyone can always specify a manual value:
INSERT INTO t (id) VALUES (999); -- always allowed with SERIAL

```

GENERATED AS IDENTITY (Modern)

Introduced in PostgreSQL 10, compliant with SQL:2003 standard.

```
-- GENERATED ALWAYS: manual inserts/updates of id are rejected
CREATE TABLE orders (
  order_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY
);

INSERT INTO orders DEFAULT VALUES; -- works, gets next value
INSERT INTO orders (order_id) VALUES (5); -- ERROR: cannot insert into column
"order_id"
-- Exception: use OVERRIDING SYSTEM VALUE
INSERT INTO orders (order_id) OVERRIDING SYSTEM VALUE VALUES (5); -- allowed

-- GENERATED BY DEFAULT: manual inserts are allowed (migration-friendly)
CREATE TABLE orders_legacy (
  order_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY
);

INSERT INTO orders_legacy (order_id) VALUES (5); -- allowed
INSERT INTO orders_legacy DEFAULT VALUES;      -- also allowed

-- Custom sequence parameters
CREATE TABLE events (
  event_id BIGINT GENERATED ALWAYS AS IDENTITY (
    START WITH 1000
    INCREMENT BY 1
    CACHE 100
    NO CYCLE
  ) PRIMARY KEY
);
```

IDENTITY vs SERIAL comparison

Feature	SERIAL	IDENTITY
SQL standard	No (PG extension)	Yes (SQL:2003)
Implicit sequence name	table_col_seq	table_col_seq
Sequence ownership	Explicit (OWNED BY)	Automatic
Prevent manual values	No	ALWAYS mode
Allow manual override	Always	BY DEFAULT mode
Permission management	Separate grant needed	Managed with column
pg_dump behavior	Includes sequence	Inline with table
ALTER COLUMN support	Limited	ALTER COLUMN SET ...

Altering IDENTITY columns

```

-- Change sequence behavior
ALTER TABLE orders ALTER COLUMN order_id SET GENERATED BY DEFAULT;
ALTER TABLE orders ALTER COLUMN order_id SET GENERATED ALWAYS;

-- Change sequence parameters
ALTER TABLE orders ALTER COLUMN order_id
    SET GENERATED ALWAYS AS IDENTITY (START WITH 5000 INCREMENT BY 2);

-- Restart sequence
ALTER TABLE orders ALTER COLUMN order_id RESTART WITH 1;
ALTER TABLE orders ALTER COLUMN order_id RESTART; -- restart from START WITH value

-- Drop identity property (column keeps its values, loses auto-increment)
ALTER TABLE orders ALTER COLUMN order_id DROP IDENTITY;
ALTER TABLE orders ALTER COLUMN order_id DROP IDENTITY IF EXISTS;

```

Sequence Ownership and Dependencies

```

-- OWNED BY links sequence lifecycle to a column
-- (sequence is dropped when the column or table is dropped)
ALTER SEQUENCE my_seq OWNED BY my_table.my_col;
ALTER SEQUENCE my_seq OWNED BY NONE; -- detach

-- Check sequence ownership
SELECT
    s.sequencename,
    s.schemaname,
    d.refobjid::regclass AS owned_table,
    a.attname AS owned_column
FROM pg_sequences s
LEFT JOIN pg_depend d ON d.objid = (
    SELECT oid FROM pg_class WHERE relname = s.sequencename AND relkind = 'S'
)
LEFT JOIN pg_attribute a ON a.attrelid = d.refobjid AND a.attnum = d.refobjsubid
WHERE s.sequencename = 'orders_order_id_seq';

```

Gaps in Sequences

Sequence gaps are normal and expected. Do not treat gaps as errors.

Gaps occur when:

1. Transactions are rolled back (sequence advances even on rollback)
2. CACHE pre-allocates values that are "lost" on server restart
3. Concurrent inserts "claim" but don't use values
4. Manual `setval()` calls

```
-- Demonstrate gap creation:
BEGIN;
SELECT nextval('order_number_seq'); -- 1000
ROLLBACK;
-- 1000 is now gone! Next nextval() returns 1001

-- With CACHE 100: server restart loses up to 99 cached values
-- Next sequence starts at last_persisted_value + 100 after restart
```

If you absolutely cannot have gaps (e.g., invoice numbers by law), you need a different approach:

```
-- Gap-free counter (uses a lock – slower, serializes inserts)
CREATE TABLE invoice_counter (
    counter_id SMALLINT PRIMARY KEY,
    next_value BIGINT NOT NULL DEFAULT 1
);
INSERT INTO invoice_counter VALUES (1, 1);

-- Get next invoice number (with row lock)
WITH counter AS (
    UPDATE invoice_counter
    SET next_value = next_value + 1
    WHERE counter_id = 1
    RETURNING next_value - 1 AS invoice_num
)
SELECT invoice_num FROM counter;
-- WARNING: This serializes all invoice inserts!
```

Resetting and Altering Sequences

```
-- Reset after truncate (common pattern)
TRUNCATE TABLE orders RESTART IDENTITY; -- resets all owned sequences

-- Or manually:
ALTER SEQUENCE orders_order_id_seq RESTART WITH 1;

-- Sync sequence to max existing value (after import/migration)
SELECT setval('orders_order_id_seq', (SELECT MAX(order_id) FROM orders));
-- Or more safely:
SELECT setval(
    'orders_order_id_seq',
    COALESCE((SELECT MAX(order_id) FROM orders), 1),
    (SELECT MAX(order_id) IS NOT NULL FROM orders)
);

-- Change sequence parameters
ALTER SEQUENCE orders_order_id_seq INCREMENT BY 2;
ALTER SEQUENCE orders_order_id_seq CACHE 100;
```

```
ALTER SEQUENCE orders_order_id_seq MAXVALUE 999999999;
ALTER SEQUENCE orders_order_id_seq CYCLE;      -- wrap to MINVALUE after MAXVALUE
ALTER SEQUENCE orders_order_id_seq NO CYCLE;    -- error at MAXVALUE
```

Multi-Tenant and Distributed ID Strategies

Strategy 1: UUID (no coordination needed)

```
id UUID DEFAULT gen_random_uuid() PRIMARY KEY
-- Pro: globally unique, no coordination
-- Con: random inserts → B-tree fragmentation (for v4)
```

Strategy 2: UUIDv7 (sequential UUID)

```
-- Snowflake-style: timestamp + random
-- Pro: sequential, globally unique, no coordination
-- Con: not yet built-in (needs extension in PG < 17)
```

Strategy 3: Snowflake-style IDs (Twitter Snowflake pattern)

64-bit ID layout:

1 bit	41 bits timestamp (ms since epoch)	10 bits node	12 bits seq
-------	------------------------------------	--------------	-------------

Bit 63: always 0 (keeps ID positive)
Bits 22-62: milliseconds since custom epoch (~69 years)
Bits 12-21: machine/datacenter ID (up to 1024 nodes)
Bits 0-11: sequence (up to 4096 IDs per ms per node)

```
-- Simple Snowflake-like ID generator in PostgreSQL
CREATE OR REPLACE FUNCTION generate_snowflake_id(node_id INT DEFAULT 1)
RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE
    epoch BIGINT := 1704067200000; -- 2024-01-01 00:00:00 UTC in ms
    ts    BIGINT;
    seq   BIGINT;
BEGIN
    ts := (EXTRACT(EPOCH FROM clock_timestamp()) * 1000)::BIGINT - epoch;
    seq := nextval('snowflake_seq') % 4096;
    RETURN (ts << 22) | ((node_id % 1024) << 12) | seq;
END;
$$;
```

Strategy 4: Per-tenant sequences

```

-- Each tenant gets their own sequence
CREATE OR REPLACE FUNCTION next_tenant_id(p_tenant_id INTEGER)
RETURNS BIGINT LANGUAGE plpgsql AS $$
DECLARE
    seq_name TEXT := format('tenant_%s_seq', p_tenant_id);
BEGIN
    -- Create sequence if it doesn't exist
    PERFORM pg_advisory_xact_lock(p_tenant_id);
    IF NOT EXISTS (SELECT 1 FROM pg_sequences WHERE sequencename = seq_name) THEN
        EXECUTE format('CREATE SEQUENCE %I START WITH 1', seq_name);
    END IF;
    RETURN nextval(seq_name);
END;
$$;

```

SQL Examples

Complete identity column patterns

```

-- Modern pattern: BIGINT IDENTITY ALWAYS
CREATE TABLE customers (
    customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Import with manual IDs (using BY DEFAULT)
CREATE TABLE customers_import (
    customer_id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- After import, sync the sequence
SELECT setval(
    pg_get_serial_sequence('customers_import', 'customer_id'),
    (SELECT COALESCE(MAX(customer_id), 0) FROM customers_import)
);

-- Shared global sequence across tables
CREATE SEQUENCE global_event_id_seq START WITH 1;

CREATE TABLE order_events (
    event_id BIGINT DEFAULT nextval('global_event_id_seq') PRIMARY KEY,
    order_id BIGINT NOT NULL,
    event_type TEXT NOT NULL,
    occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```



```
CREATE TABLE payment_events (  
  event_id    BIGINT DEFAULT nextval('global_event_id_seq') PRIMARY KEY,  
  payment_id  BIGINT NOT NULL,  
  event_type  TEXT    NOT NULL,  
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
-- Both tables share one sequence → event_ids are globally unique across both tables
```

Common Mistakes

1. Using `currval()` in a different session

```
-- WRONG: currval() only works after nextval() in the SAME session  
-- This fails if called in a different connection  
SELECT currval('my_seq'); -- ERROR: currval not yet defined in this session  
  
-- RIGHT: use RETURNING clause  
INSERT INTO orders (customer_id) VALUES (1) RETURNING order_id;
```

2. Not syncing sequence after data import

```
-- After COPY or bulk INSERT with explicit IDs:  
-- Next AUTO INSERT will fail with duplicate key if sequence < max(id)  
-- Fix:  
SELECT setval(  
  pg_get_serial_sequence('orders', 'order_id'),  
  (SELECT MAX(order_id) FROM orders)  
);
```

3. **Treating sequence gaps as errors or data corruption** — Gaps are normal. Never write code that assumes sequential IDs.
4. **CACHE too high in multi-master or failover setups** — With `CACHE 100`, a failover can "lose" up to 99 IDs. Consider `CACHE 1` for strict monotonicity (at performance cost).
5. **Using SERIAL for new code (it's legacy)** — Use `GENERATED ALWAYS AS IDENTITY` for all new tables.

Best Practices

1. Use `BIGINT GENERATED ALWAYS AS IDENTITY` for all new tables.
 2. Use `GENERATED BY DEFAULT` only during migrations where you need to specify IDs.
 3. Always use `RETURNING id` after `INSERT` to get the generated value — never use `currval()`.
 4. Set `CACHE` to `1` for sequences where gap minimization matters; higher cache for performance.
 5. After bulk imports, sync sequences with `setval()` to the current MAX value.
 6. Name sequences explicitly in the `GENERATED AS IDENTITY (...)` block when you need custom parameters.
 7. Use **UUIDs for distributed systems** where multiple nodes generate IDs independently.
-

Performance Considerations

Sequence performance characteristics:

<code>nextval()</code> call:	~microseconds (very fast, non-blocking)
CACHE = 1:	1 WAL write per <code>nextval()</code> call
CACHE = 100:	1 WAL write per 100 <code>nextval()</code> calls → 100x less WAL
CACHE > 1:	may lose cached values on crash/restart

For high-throughput insert workloads:

CACHE 50-1000:	significantly reduces sequence-related WAL overhead
CACHE 1:	maximum safety, slowest for high-concurrency inserts

```
-- Check sequence performance with high concurrency
-- Simulate 1000 concurrent nextval calls
EXPLAIN (ANALYZE, BUFFERS)
SELECT nextval('orders_order_id_seq')
FROM generate_series(1, 1000);
-- Sequences are designed for this – should be sub-millisecond total
```

Interview Questions & Answers

Q1: Is a sequence operation transactional in PostgreSQL?

A: No. `nextval()` is intentionally non-transactional. If a transaction calls `nextval()` and then rolls back, the value is not "returned" to the sequence. This is by design — if sequences were transactional, they would need to block concurrent transactions waiting to use the same values, destroying their concurrency benefits. The trade-off is sequence gaps.

Q2: What is the difference between SERIAL and GENERATED AS IDENTITY ?

A: Both auto-generate sequential integers. `SERIAL` is a PostgreSQL-specific macro creating a sequence with a column default. `GENERATED AS IDENTITY` is SQL standard (SQL:2003), supports `ALWAYS` mode (rejects manual values), has integrated permission management, and is handled correctly by `pg_dump`. Use `GENERATED AS IDENTITY` for all new code.

Q3: How do you get the generated ID after an INSERT?

A: Use the `RETURNING` clause: `INSERT INTO orders ... RETURNING order_id`. Do NOT use `curval()` — it only works within the same session after `nextval()`, is not available via JDBC/most drivers, and is fragile in application code.

Q4: Why do sequences have gaps and why is that acceptable?

A: Sequences advance even for rolled-back transactions. This is necessary for non-blocking concurrent ID generation — if sequences were transactional, each `INSERT` would need to wait for the previous one to commit. Gaps are acceptable because: (1) IDs are identifiers, not accounting records, (2) The ID value itself carries no meaning other than uniqueness, (3) Having gaps does not indicate missing data.

Q5: What is sequence cache and what are its trade-offs?

A: Cache pre-allocates N sequence values per session, stored in memory. With `CACHE 100`, a session grabs 100 values at once (1 WAL write) vs. 100 individual grabs (100 WAL writes). Trade-off: cached values that are never used (e.g., session ends, server restarts) create gaps. For maximum performance: high cache. For gap minimization: `CACHE 1`.

Q6: How do you reset a sequence after truncating a table?

A: `TRUNCATE TABLE orders RESTART IDENTITY;` — this truncates and resets all owned sequences atomically. Or: `ALTER SEQUENCE orders_order_id_seq RESTART WITH 1;` .

Q7: What happens when a sequence reaches its MAXVALUE?

A: If `NO CYCLE` (default): the next `nextval()` call raises an error. If `CYCLE`: the sequence resets to `MINVALUE`. For a `BIGINT` sequence with `START 1` and `CACHE 1`, you'd need 9.2×10^{18} rows before this is a problem.

Q8: How do you safely backfill IDs when migrating from an external system?

A: Use `GENERATED BY DEFAULT AS IDENTITY` to allow manual ID specification. After the migration, sync the sequence: `SELECT setval(pg_get_serial_sequence('table', 'col'), (SELECT COALESCE(MAX(id), 0) FROM table));` . Then optionally switch to `GENERATED ALWAYS` mode.

Exercises with Solutions

Exercise 1

Create a custom sequence for invoice numbers (format: INV-YYYY-#####) that starts at 1 each year and generates the formatted string.

Solution:

```
CREATE SEQUENCE invoice_seq_2024 START WITH 1;

CREATE OR REPLACE FUNCTION next_invoice_number(p_year INT DEFAULT EXTRACT(YEAR FROM
now()))::INT)
RETURNS TEXT LANGUAGE plpgsql AS $$
DECLARE
    v_seq_name TEXT := format('invoice_seq_%s', p_year);
    v_num       BIGINT;
BEGIN
    -- Create sequence for this year if it doesn't exist
    IF NOT EXISTS (SELECT 1 FROM pg_sequences WHERE sequencename = v_seq_name) THEN
        EXECUTE format('CREATE SEQUENCE %I START WITH 1', v_seq_name);
    END IF;
    v_num := nextval(v_seq_name);
    RETURN format('INV-%s-%05d', p_year, v_num);
END;
$$;

SELECT next_invoice_number();      -- INV-2024-00001
SELECT next_invoice_number();      -- INV-2024-00002
SELECT next_invoice_number(2025); -- INV-2025-00001
```

Exercise 2

Write a migration script to convert all SERIAL columns to GENERATED AS IDENTITY in a table.

Solution:

```
-- Step 1: Drop the default (breaks the SERIAL link)
ALTER TABLE orders ALTER COLUMN order_id DROP DEFAULT;

-- Step 2: Drop the old sequence (if owned by column, it will be recreated)
-- Note: identity columns create their own sequence
DROP SEQUENCE IF EXISTS orders_order_id_seq;

-- Step 3: Add IDENTITY behavior
ALTER TABLE orders ALTER COLUMN order_id
    ADD GENERATED ALWAYS AS IDENTITY (
        START WITH 1
        RESTART WITH 1
    );

-- Step 4: Sync to current max
ALTER TABLE orders ALTER COLUMN order_id
    RESTART WITH (SELECT COALESCE(MAX(order_id), 0) + 1 FROM orders);
```

Cross-References

- `01_numeric_types.md` — BIGINT for sequence values
- `04_boolean_uuid_types.md` — UUID as alternative to sequences
- `08_constraints.md` — PRIMARY KEY uses sequences
- `../09_Transactions_Concurrency/` — sequence non-transactionality
- `../12_Production_PostgreSQL/` — sequence monitoring and exhaustion alerts